
Phase-Adaptive Generation: Learning Block Budgets to Reduce Diffusion-LM Calls

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Diffusion language models can revise several tokens per forward call, but their
2 practical efficiency depends on deciding how large a span to attempt and when to
3 stop refining it. Phase-Adaptive Generation (PAG) learns this refinement budget
4 from prior block trajectories while retaining reactive delimiter boundaries. On
5 LLaDA-8B-Instruct, PAG uses 6.7% fewer model calls on MATH-500 while accu-
6 racy is 1.33 percentage points lower; on GSM8K it saves 4.0% while accuracy
7 is 1.60 percentage points lower. PAG obtains this trade-off by separating two de-
8 cisions: recent blocks predict how much refinement to spend next, while current
9 predictions still choose where the block ends. We then test how far this gain can
10 be extended. After normalizing each model–dataset setting to its AdaBlock base-
11 line, a rule shared by LLaDA and Dream uses 9.6% fewer calls overall, although
12 Dream does not meet our accuracy requirement. Stricter rules preserve outputs
13 and save up to 1.24%. Batched checking suggests still larger call reductions, with
14 exact matching unresolved in 22 of 64 pilot prompts. Overall, phase history is
15 a useful signal for allocating compute; current boundaries and reference checks
16 provide the safeguards needed to use it reliably.

17 1 Introduction

18 Diffusion language models generate by repeatedly revising masked positions, so one model call can
19 advance several tokens at once. This parallelism creates a control problem absent from left-to-right
20 decoding: how large a span should the model attempt, and how long should it refine that span before
21 moving on? Existing decoders answer with fixed schedules or current-step confidence signals [Wu
22 et al., 2026, Lu et al., 2026, Mohamed et al., 2025]. Both are useful, but neither asks whether the
23 trajectory itself forecasts where later compute will be needed.

24 Phase-adaptive generation (PAG) starts from a simple observation: reasoning traces are not uni-
25 formly difficult. Template text and delimiters often stabilize early, while arithmetic, variable bind-
26 ing, and answer finalization remain uncertain for longer. The difficult part is turning this structure
27 into a decoder. Offline accuracy is not enough. The scheduler changes the trajectory that produces
28 its own future features; block sizes are strongly multimodal; and a premature commitment can alter
29 every later block. A useful predictor can therefore be a poor controller.

30 We develop the method from trace analysis through deployment. We first instrument Dream and
31 LLaDA [Ye et al., 2025, Nie et al., 2025], measure when tokens stop changing, and use change-
32 point detection (CPD) to find local shifts in uncertainty. Because the detected boundaries vary with
33 the detector settings, we train instead on 138,901 block records collected by running AdaBlock on
34 5,000 GSM8K training problems. During generation, PAG records each completed block’s size,

confidence, and number of model calls. A small Transformer reads the latest eight block records and predicts how many refinement calls to allow for the next block. If recent blocks have stabilized quickly, it can assign a smaller budget; if they have needed more refinement, it retains more calls. AdaBlock still decides where the next block ends from the model’s current predictions. The later evaluations test whether this design transfers across models, preserves outputs under stricter rules, and benefits from batched checking.

Across the experiments, counting every proposal and refinement call shows that PAG reduces the mean number of forward evaluations (NFE) on LLaDA by 4.0% on GSM8K and 6.7% on MATH-500, with accuracy decreases of 1.60 and 1.33 percentage points. The gain comes from splitting the scheduling decision in two. AdaBlock still uses the model’s current predictions to decide where each block ends. PAG changes only how many refinement calls the next block receives, using the last eight completed blocks as context. Predictable stretches can therefore stop earlier without letting the history model change the span being decoded. The later runs test how safely the gain extends: after dividing each setting by its AdaBlock baseline, the cross-model rule reduces NFE by 9.6% overall; output-preserving variants save up to 1.24%, and batched checking produces larger raw reductions. The main result is a clear efficiency–quality trade-off on LLaDA, together with direct evidence about the safeguards needed to extend it.

Contributions. We make three contributions. First, we introduce PAG, a hybrid diffusion-LM scheduler that uses the last eight completed blocks to set the next refinement budget while leaving AdaBlock in control of block boundaries. When every model call is counted, PAG reduces LLaDA NFE by 4.0% on GSM8K and 6.7% on MATH-500, with accuracy decreases of 1.60 and 1.33 percentage points. Second, we connect token-stabilization analysis and 138,901 block records to a practical scheduler in which recent refinement behavior changes the next budget. A size-conditioned control separates this history signal from information already available in the current block. Third, seven artifact-backed protocols across LLaDA and Dream test how far the mechanism extends through cross-model transfer, exact output preservation, and batched verification. They lead to a concrete design rule: use current predictions for block boundaries, history for bounded budget adjustments, and an audited reference transition for token acceptance.

2 Related work

Masked and block diffusion language models. Discrete and masked diffusion models replace left-to-right factorization with iterative denoising [Lou et al., 2024, Sahoo et al., 2024]. LLaDA and Dream scale this formulation to instruction following and reasoning [Nie et al., 2025, Ye et al., 2025]. Block diffusion introduces a semi-autoregressive interface: the decoder completes one contiguous block before exposing it as context to the next [Arriola et al., 2025]. That interface makes block size and within-block refinement explicit controls. Our work studies whether realized block histories help set those controls; it does not change the model or training objective.

Adaptive and early-exit decoding. Fast-dLLM combines confidence transfer with key–value (KV) caching and parallel decoding [Wu et al., 2026]. AdaBlock chooses semantic boundaries from delimiter confidence and adapts refinement to the active block [Lu et al., 2026]; it is our native decoder and strongest same-stack baseline. Adaptive Parallel Decoding (APD) learns token-wise denoising paths, whereas SchED and more recent stability rules stop from inter-step similarity or persistent predictions [Israel et al., 2025, Mohamed et al., 2025, 2026]. These methods establish that current-state confidence and stability are useful. PAG asks a different question: whether previous blocks add predictive value after controlling for a strong current-state or size-conditioned rule.

Risk control and verified acceleration. Learn-then-Test and conformal risk control provide finite-sample tools for selecting a policy under a bounded loss [Angelopoulos et al., 2025, 2024]. We apply this machinery to a binary loss that records whether early stopping changes a block relative to full refinement; task correctness is evaluated separately. Self-speculative diffusion-LM decoders instead batch draft states and verify them against a reference transition [Gao et al., 2025, Han et al., 2026]. Our later protocols combine these ideas but also audit a systems assumption that is easy to omit: a nominally deterministic transition must remain invariant when evaluated alone or as one row of a batch. What remains unclear in this literature is whether a predictive phase signal survives both

closed-loop deployment and the batch implementation of verification. Our contribution is the resulting end-to-end evidence boundary, not a claim of superiority over these contemporaneous decoders.

3 Phase-adaptive generation

PAG has three stages (Figure 1). First, offline trace analysis tests whether different parts of a response need different amounts of refinement. Second, completed AdaBlock runs train a predictor for the next block’s call budget. Third, during generation, PAG records the size, confidence, and call count of each completed block and feeds the latest eight records to the predictor. Recent blocks that stabilized quickly lead to a smaller budget; blocks that needed more refinement lead to a larger one. AdaBlock continues to choose the block boundary from the model’s current predictions. This is how PAG cuts model calls without allowing the history predictor to choose the generated span, and the division of responsibilities was fixed before the audited evaluation.

3.1 Phase signals are diagnostic, not labels

For each output position, we record its predicted identity, top-one and top-two probabilities, and entropy after every refinement call. The stabilization time of token i is the earliest step after which its identity never changes,

$$s_i = \min\{t : \hat{x}_i^{(u)} = \hat{x}_i^{(T)} \text{ for all } u \geq t\}. \quad (1)$$

We read confidence, margin, entropy, and step index at the stabilization time in Equation 1, then segment each feature along token position with Pruned Exact Linear Time (PELT) [Killick et al., 2012]. Figure 2 shows the recurring pattern: long high-confidence spans are interrupted by lower-confidence computation, and block sizes concentrate on delimiter-only and default-cap modes.

The segmentation is intentionally not used as supervision. Boundaries move when the feature, smoothing window, cost, or penalty changes; representative traces yield between two and eight segments under reasonable settings. CPD therefore establishes that trajectories are structured, but does not define a stable scheduler target. The training path instead uses decisions directly observed from a block decoder.

3.2 Block-native next-control prediction

We run AdaBlock with LLaDA-8B-Instruct on 5,000 GSM8K training problems [Cobbe et al., 2021]. After each decoded block, we store its size and realized NFE, stabilization/refinement statistics, mean and minimum confidence, digit fraction, delimiter fraction, and gap statistics. The corpus contains 138,901 training tuples and 5,543 validation tuples. Sliding windows of eight realized tuples predict the next pair $y_t = (b_t, r_t)$, where b_t is block size and r_t is the refinement budget.

The target distribution rules out ordinary regression. A squared-error predictor averages the size-1 delimiter mode and the content-block mode, producing boundaries that seldom occur in the trace corpus. We instead use a Transformer encoder [Vaswani et al., 2017] with a 128-way categorical head for b_t and an 83-threshold ordinal head for r_t . The budget head is conditioned on the block-size logits. With categorical logits z^b and ordinal logits z^r , training minimizes

$$\mathcal{L} = \text{CE}(z^b, b_t) + \frac{1}{83} \sum_{k=1}^{83} \text{BCEWithLogits}(z_k^r, \mathbf{1}[r_t > k]). \quad (2)$$

Here, CE is categorical cross-entropy, and BCEWithLogits is binary cross-entropy applied to logits. The model has hidden size 64, four heads, four layers, 0.1 dropout, and learning rate 5×10^{-4} . Inputs are standardized with the training-set mean and standard deviation. Appendix B reports regression, random-forest, and architecture ablations.

3.3 Hybrid online scheduling

At block t , AdaBlock scans the model’s current token probabilities for a high-confidence delimiter and returns the boundary b_t . The first block follows AdaBlock; later blocks pass the last eight realized tuples to the predictor and use only its budget r_t . Within the active block, masked positions

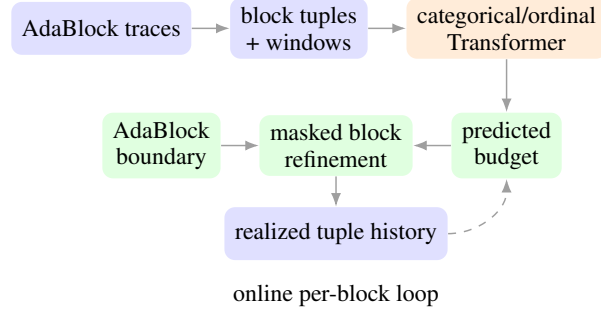


Figure 1: PAG separates two control signals. Current model predictions determine the next semantic boundary; history predicts a refinement budget. The realized tuple closes the loop only after the block is decoded.

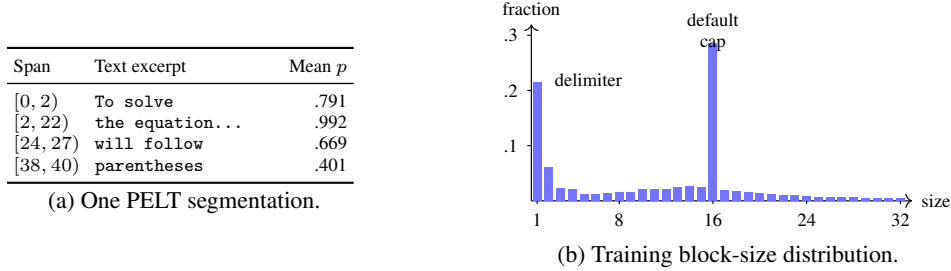


Figure 2: From token-level phases to block-native data. (a) One algebra trace contains adjacent spans with markedly different stabilization confidence; the exact boundaries vary with detector settings. (b) The 138,901 training blocks form a discrete, multimodal distribution. Size 1 is dominated by delimiter blocks, while size 16 is the default content cap.

are refined until the budget is reached. The decoder may then exit when all remaining predictions exceed confidence 0.8 or have stayed unchanged for two calls; a hard ceiling prevents unbounded refinement. Each initial proposal call and each refinement call counts toward NFE.

Letting the predictor choose both controls often selects $(b_t, r_t) = (1, 1)$. This trivial tuple then encourages another small prediction, causing the online features to drift from the training data. Retaining AdaBlock’s delimiter boundary breaks the feedback loop and isolates the experimental question: how does history-conditioned budgeting compare with AdaBlock and a control conditioned only on the current block size b_t ?

4 Evidence protocol

Initial audited evaluation. The primary PAG comparison uses deterministic LLaDA-8B-Instruct decoding on all 1,319 GSM8K test problems and a 300-example index-stratified MATH-500 sample [Hendrycks et al., 2021]. GSM8K also includes a frozen development slice for choosing among eight scheduler variants and a disjoint 919-example confirmatory complement. The comparators are AdaBlock and a history-free lookup policy that returns the median training NFE for the current block size. Every method counts the boundary proposal call and all subsequent refinement calls. The legacy course counter omitted the initial proposal, so those earlier values are excluded from every claim here.

Cross-model evaluation. To test whether the stopping rule transfers across model families, we add a risk check and call the resulting policy risk-calibrated PAG (RC-PAG). At each possible stop, a small predictor estimates whether committing the current block would differ from continuing the same decoder state to full refinement. RC-PAG stops only when this risk is below a calibrated threshold; otherwise, it continues refining. We use fixed LLaDA-8B-Instruct and Dream-v0-Instruct-

152 7B checkpoints, temperature zero, generation length 256, at most 64 refinement calls, and block
153 length at most 64.

154 Separate mixtures of GSM8K, MATH, and Mostly Basic Python Problems (MBPP) training data are
155 used to train the risk predictor, choose among stopping rules, and calibrate the selected rule [Austin
156 et al., 2021]. Confirmation uses 500 GSM8K test, 300 MATH-500, 100 sanitized MBPP, and 100
157 HumanEval prompts [Chen et al., 2021]; HumanEval is outside the calibration domain. During
158 calibration, each proposed stop is paired with a reference continuation that starts from the same
159 decoder state and runs to full refinement. The prompt-level loss is one if any early-committed block
160 differs from its reference and zero otherwise. Exact binomial Learn-then-Test selects among the
161 candidate rules while limiting the probability of any false selection across the family to 0.05. This
162 statistical guarantee bounds the declared disagreement loss, not answer correctness.

163 **Criteria for a confirmed result.** Results are paired by prompt. We report mean NFE, exact task
164 accuracy, and latency, with 10,000-sample paired bootstrap intervals where available. Accuracy
165 differences are absolute percentage-point changes: moving from 38% to 37%, for example, is a
166 decrease of one percentage point. A cross-model policy counts as a confirmed extension only if it
167 reduces NFE on both models, beats the best eligible rule that does not use learned history, and has
168 an accuracy-difference lower bound of at least -2 percentage points. Later protocols also require a
169 minimum saving on each model or exact agreement with the reference output. For batched protocols,
170 one batched forward call counts as one physical NFE; the number of rows evaluated within that call
171 is reported separately.

172 5 Results

173 Figure 3 organizes the evidence around the central result: PAG reduces LLaDA calls by up to
174 6.7%. The remaining stages test whether that gain transfers across models, survives exact-output
175 constraints, and can be enlarged through verification. Predeclared criteria keep these extensions
176 separate from the confirmed headline result.

177 5.1 PAG saves up to 6.7% NFE by adapting refinement budgets

178 Table 1 gives the LLaDA result. Relative to AdaBlock, PAG saves 4.0% NFE on GSM8K and
179 6.7% on MATH-500, while accuracy is 1.60 and 1.33 percentage points lower. On the untouched
180 GSM8K complement, the paired accuracy difference is -1.20 percentage points, with a 95% confi-
181 dence interval of $[-2.61, 0.22]$ percentage points. This is the paper’s main efficiency–quality result:
182 history-conditioned budgets consistently reduce calls on both reasoning benchmarks.

183 The size-conditioned control uses 6.5% less NFE than AdaBlock on GSM8K, confirming that the
184 current block provides a useful scheduling prior. PAG addresses a different source of information:
185 how the preceding blocks actually refined. This distinction motivates combining a simple current-
186 block prior with PAG’s history-dependent budget adjustment. On the synchronized 150-prompt
187 timing subset, both decoders reach 84% accuracy and PAG is 0.31 seconds faster on average; the
188 paired latency interval crosses zero, so we treat this timing result as supporting rather than conclusive
189 evidence.

190 5.2 Cross-model calibration separates transferable and model-specific gains

191 The cross-model experiment showed that phase history remained predictive beyond the original
192 scheduler. On held-out prompts, the history-based risk predictor reached an area under the receiver
193 operating characteristic curve (AUROC; higher is better) of 0.982 and a Brier score (lower is better)
194 of 0.051 for LLaDA; the corresponding Dream values were 0.990 and 0.026. All candidate stopping
195 rules chosen before calibration also passed their finite-sample tests for full-refinement disagreement.
196 Table 2 measures how this predictive signal translated into end-to-end accuracy and compute.

197 To compare settings with different baseline costs, we divide each policy’s NFE by AdaBlock’s NFE
198 on the same model–dataset pair and average the four ratios. This normalized NFE falls from 1.000
199 to 0.904, compared with 0.968 for the best eligible rule without learned history. On LLaDA, the
200 policy preserves GSM8K accuracy and lowers MATH-500 accuracy by only 0.4 percentage points
201 while saving 2.1% and 1.3% NFE. Dream provides larger call reductions, but its accuracy changes

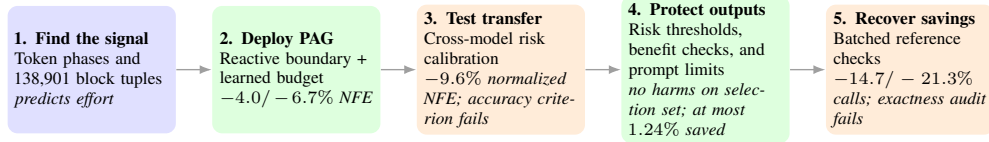


Figure 3: Progression from phase prediction to guarded acceleration; savings are relative to AdaBlock. PAG’s strongest result is a 6.7% NFE reduction on MATH-500 while accuracy is 1.33 percentage points lower. Later stages measure cross-model transfer, output preservation, and the potential of batched verification.

Table 1: LLaDA evaluation. Accuracy is exact-answer percentage (higher is better); NFE counts all model forward calls (lower is better). The lookup baseline conditions only on block size.

Dataset	Method	Accuracy $\uparrow$	Mean NFE $\downarrow$	Δ NFE
GSM8K ($n = 1,319$)	AdaBlock	77.79	87.81	—
	size lookup	76.42	82.14	−6.5%
	PAG	76.19	84.26	−4.0%
MATH-500 ($n = 300$)	AdaBlock	38.00	119.65	—
	PAG	36.67	111.67	−6.7%

202 exceed the predeclared limit; the lowest lower endpoint among the accuracy confidence intervals is
 203 −21.2 percentage points, so the joint policy does not qualify as a confirmed cross-model extension.
 204 The experiment therefore separates a conservative transferable gain on LLaDA from an aggressive,
 205 model-specific Dream setting.

206 5.3 Conservative routing preserves outputs

207 The next four protocols prioritize exact output preservation. A local risk-threshold policy stops
 208 only when the current block’s predicted risk falls below a threshold; it saves 9.1%/3.2% NFE on
 209 LLaDA/Dream, but Dream falls below the predeclared 5% savings floor. A benefit-and-harm policy
 210 adds predictors for both the expected saving and the chance of changing the answer. Its harm AU-
 211 ROC is 0.372/0.456, which is too low to use this decision rule. A prompt-level policy instead limits
 212 the cumulative predicted risk and number of early stops over an entire prompt; a relaxed version
 213 allows larger limits. On the held-out sets used to choose candidates, these prompt-level policies
 214 record zero harmful regressions while saving 0.75%/0.45% and 1.24%/0.65%. These runs establish
 215 that the learned signal can preserve observed outputs, with a smaller gain than the main PAG setting.

216 5.4 Batched verification points to larger call reductions

217 The batched-reference pilot checked several fixed numbers of future refinement steps together in
 218 one model call. This lowered physical invocations from 83.47 to 71.19 on LLaDA and from 97.63
 219 to 76.81 on Dream, reductions of 14.7% and 21.3%. These were the largest raw call reductions
 220 in the progression and showed the potential of moving several checks into one invocation. The
 221 implementation was not lossless: 22 of 64 prompt trajectories disagreed with AdaBlock, and LLaDA
 222 latency rose from 3.09 to 3.83 seconds because each invocation evaluated more rows. A subsequent
 223 equivalence preflight isolated the issue: identical model and software settings still produced different
 224 next decoder states when the same reference step was evaluated alone or in batches of different depth.
 225 Figure 4 shows this audit. The path to larger safe reductions is therefore specific: learned histories
 226 propose budgets, and a reference step that is unchanged by batching retains acceptance authority.

227 6 Discussion

228 **Phase history is a useful budget signal.** Trace analysis, next-block prediction, and deployed
 229 LLaDA results give the same answer: previous refinement behavior helps estimate how much work
 230 the next block will need. PAG turns that signal into 4.0% and 6.7% fewer calls by changing only
 231 the budget. The cross-model experiments further show that history is most reliable as a guide to

Table 2: Cross-model confirmation for the risk-calibrated history policy. ΔAcc is the accuracy change in percentage points and ΔNFE is relative to AdaBlock.

Model	Dataset	AdaBlock Acc./NFE	RC-PAG Acc./NFE	ΔAcc	ΔNFE
LLaDA	GSM8K	78.2 / 86.89	78.2 / 85.06	+0.0	-2.1%
LLaDA	MATH-500	37.7 / 119.72	37.3 / 118.17	-0.4	-1.3%
Dream	GSM8K	69.8 / 103.04	53.6 / 89.46	-16.2	-13.2%
Dream	MATH-500	45.3 / 123.56	40.3 / 114.39	-5.0	-7.4%

compute allocation, while final token decisions remain tied to current model evidence [Lu et al., 2026, Mohamed et al., 2025, 2026].

Block size and phase history provide different signals. Block size reflects both span length and AdaBlock’s boundary decision, while phase history records how preceding blocks stabilized. The size-conditioned control confirms that the current block is a useful, low-cost prior. A natural next version of PAG would start from that prior and let history make a bounded adjustment, retaining PAG’s ability to react to how the preceding blocks actually behaved.

Verification provides a path to larger gains. The batched pilot reduces physical calls by 14.7% on LLaDA and 21.3% on Dream, showing the potential of checking several proposed steps together. Realizing those gains safely requires the batched and serial transitions to produce the same tokens, cache state, attention mask, boundary, and complete next state. This batch-invariance requirement complements token-level verification in self-speculative decoders [Gao et al., 2025, Han et al., 2026] and gives a concrete engineering target. Physical calls, evaluated rows, latency, and memory should still be reported separately.

7 Limitations

The study covers two 7–8B model families, 256-token generations, and math/code benchmarks; it does not show that phase history is unhelpful for larger models, longer contexts, or a scheduler trained jointly with the model. CPD evidence is qualitative and detector-dependent. The statistical guarantees apply to the declared full-refinement disagreement loss and calibration mixture, not semantic correctness or arbitrary deployment data. Code-task exact-match rates are sensitive to extraction and test harnesses, so they appear in the appendix rather than drive the main claim. Latency was measured at batch size one on a single-GPU workflow and is not a serving benchmark. Finally, the batched-pilot and equivalence-preflight results diagnose this implementation of batched verification; they do not rule out verified acceleration with stricter batch-invariant kernels or serial verification.

8 Conclusion

PAG shows that recent refinement history can guide a diffusion language model’s compute budget without retraining the model. AdaBlock chooses each block boundary from the current prediction, and PAG uses the last eight completed blocks only to set the next refinement budget. This simple division reduces LLaDA model calls by 4.0% on GSM8K and 6.7% on MATH-500, while accuracy is 1.60 and 1.33 percentage points lower. The size-controlled analysis identifies current block size as a useful prior, while the cross-model and output-preserving runs clarify when richer history remains useful.

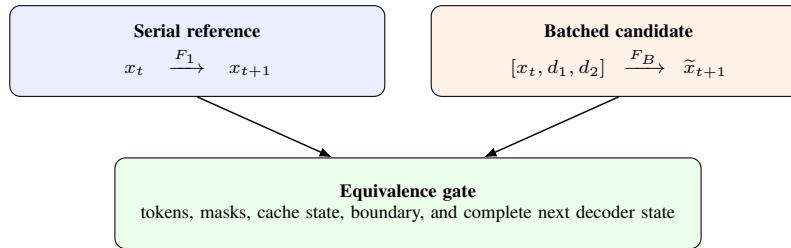
The later protocols point to a practical next design: begin with a strong current-state or size-based budget, let history make a bounded adjustment, and place final token acceptance behind a batch-invariant reference check. Conservative prompt-level risk limits already preserve observed outputs while saving up to 1.24%, and batched verification shows raw call reductions of 14.7% and 21.3% once exact transition matching is resolved. This architecture retains PAG’s headline efficiency benefit and gives a concrete route to larger, reliable reductions.

References

- Anastasios N. Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. Conformal risk control. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2208.02814>.
- Anastasios N. Angelopoulos, Stephen Bates, Emmanuel J. Candès, Michael I. Jordan, and Lihua Lei. Learn then test: Calibrating predictive algorithms to achieve risk control. *The Annals of Applied Statistics*, 19(2):1641–1662, 2025. doi: 10.1214/24-AOAS1998. URL <https://arxiv.org/abs/2110.01052>.
- Marianne Arriola, Aaron Gokaslan, Justin T. Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2503.09573>.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021. URL <https://arxiv.org/abs/2108.07732>.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, et al. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
- Yifeng Gao, Ziang Ji, Yuxuan Wang, Biqing Qi, Hanlin Xu, and Linfeng Zhang. Self speculative decoding for diffusion large language models, 2025. URL <https://arxiv.org/abs/2510.04147>.
- Ligong Han, Hao Wang, Han Gao, Kai Xu, and Akash Srivastava. S2D2: Fast decoding for diffusion LLMs via training-free self-speculation, 2026. URL <https://arxiv.org/abs/2603.25702>.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset, 2021. URL <https://arxiv.org/abs/2103.03874>.
- Daniel Israel, Guy Van den Broeck, and Aditya Grover. Accelerating diffusion LLMs via adaptive parallel decoding. In *Advances in Neural Information Processing Systems*, volume 38, 2025. doi: 10.52202/085713-1763. URL <https://arxiv.org/abs/2506.00413>.
- Rebecca Killick, Paul Fearnhead, and Idris A. Eckley. Optimal detection of changepoints with a linear computational cost. *Journal of the American Statistical Association*, 107(500):1590–1598, 2012. doi: 10.1080/01621459.2012.737745. URL <https://arxiv.org/abs/1101.1438>.
- Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 32819–32848. PMLR, 2024. URL <https://arxiv.org/abs/2310.16834>.
- Guanxi Lu, Hao Mark Chen, Yuto Karashima, Zhican Wang, Daichi Fujiki, and Hongxiang Fan. AdaBlock-dLLM: Semantic-aware diffusion LLM inference via adaptive block size. In *International Conference on Learning Representations*, 2026. URL <https://arxiv.org/abs/2509.26432>.
- Amr Mohamed, Yang Zhang, Michalis Vazirgiannis, and Guokan Shang. Fast-decoding diffusion language models via progress-aware confidence schedules, 2025. URL <https://arxiv.org/abs/2512.02892>.

- 320 Amr Mohamed, Guokan Shang, and Michalis Vazirgiannis. LESS is more: Mutual-stability sam-
 321 pling for diffusion language models, 2026. URL <https://arxiv.org/abs/2606.16908>.
- 322 Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-
 323 Rong Wen, and Chongxuan Li. Large language diffusion models, 2025. URL <https://arxiv.org/abs/2502.09992>.
- 325 Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T.
 326 Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language
 327 models. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/
 328 079017-4135. URL <https://arxiv.org/abs/2406.07524>.
- 329 Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez,
 330 Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Infor-*
 331 *mation Processing Systems*, volume 30, 2017. URL <https://arxiv.org/abs/1706.03762>.
- 332 Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song
 333 Han, and Enze Xie. Fast-dLLM: Training-free acceleration of diffusion LLM by enabling KV
 334 cache and parallel decoding. In *International Conference on Learning Representations*, 2026.
 335 URL <https://arxiv.org/abs/2505.22618>.
- 336 Jiacheng Ye, Zihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng
 337 Kong. Dream 7B: Diffusion large language models, 2025. URL <https://arxiv.org/abs/2508.15487>.

339 A Verification equivalence audit



340

Figure 4: Exactness audit for batched reference checks. A batched row is eligible only when it produces the same complete next decoder state as the serial reference step. The pilot disagrees on 22 of 64 prompts, and the subsequent preflight rejects the depth-dependent batch shape before rollout.

341 B Predictor and integration diagnostics

342 **Continuous regression.** The first Transformer predicts real-valued controls with validation loss
 343 0.096 after target normalization. Deployed on 30 arithmetic prompts, it reaches 73% accuracy versus
 344 93% for AdaBlock and averages 8.1 blocks rather than 12.3. Inspection shows interpolated block
 345 sizes merging semantic spans and producing repeated or truncated text. This run motivated the
 346 categorical block head; it is not used as a baseline in the audited evaluation.

347 **Random forest.** A 200-tree, depth-15 multi-output forest uses rolling statistics from a five-tuple
 348 window. On 90,994 training and 22,907 held-out examples, block-size mean absolute error (MAE)
 349 is 7.24 and stabilization-step MAE is 1.66. The last block size is the largest individual feature
 350 (importance 0.214), supporting the explicit size-lookup control.

351 **Transformer ablation.** The categorical/ordinal sweep varies context length, hidden size, attention
 352 heads, layers, dropout, and learning rate. The selected model uses an eight-block context, hidden
 353 size 64, four attention heads, and four layers; it reaches validation loss 2.216957. Wider models do
 354 not improve this objective: at four layers, the best losses for widths 32, 64, 128, 256, and 512 are
 355 2.233, 2.217, 2.238, 2.289, and 2.336.

C Complete cross-model confirmation

Table 3 reports every dataset/model pair from the cross-model confirmation; the code tasks are retained for completeness but do not drive the main claim.

Table 3: Complete cross-model confirmation. Accuracy is percentage and NFE is the mean number of physical model calls. HumanEval is declared out of domain.

Model	Dataset	Method	Accuracy	Mean NFE
LLaDA	GSM8K	AdaBlock / RC-PAG	78.2 / 78.2	86.89 / 85.06
LLaDA	MATH-500	AdaBlock / RC-PAG	37.7 / 37.3	119.72 / 118.17
LLaDA	MBPP	AdaBlock / RC-PAG	4.0 / 4.0	71.66 / 70.57
LLaDA	HumanEval	AdaBlock / RC-PAG	32.0 / 33.0	95.48 / 94.39
Dream	GSM8K	AdaBlock / RC-PAG	69.8 / 53.6	103.04 / 89.46
Dream	MATH-500	AdaBlock / RC-PAG	45.3 / 40.3	123.56 / 114.39
Dream	MBPP	AdaBlock / RC-PAG	3.0 / 0.0	80.02 / 54.25
Dream	HumanEval	AdaBlock / RC-PAG	48.0 / 16.0	84.78 / 64.50

D Artifact-backed protocol ledger

Table 4 records the evidence source for each reported phase. The v1 and v4–v9 labels are internal configuration identifiers retained only to connect claims to released artifacts; they do not denote different model architectures. Configurations v2–v3 were superseded before full evaluation. Hashes are shortened only for readability. All records are non-mock and pin model and dataset revisions in their environment manifests.

Table 4: Artifact provenance for numerical claims.

Evidence	Config/run hash	Authoritative artifact
Initial PAG	Strategy 1	regraded report summarized in final report
RC-PAG v1	5197d981c0bf	report/summary.json, claim_audit.json
Local risk v4	8cce1af8e35b	screening_summary.json
Advantage v5	7688c5235bd4	screen records, advantage_manifest.json
Risk ledger v6	58e76c3eb9b1	readiness_audit.json
Relaxed gate v7	c1eda289fb08	readiness_audit.json
Verified pilot v8	d36b982c2388	parity_audit.json
Equivalence audit v9	588204cfb482	equivalence_audit.json

E Accounting and reporting rules

NFE denotes physical invocations of the model forward function, including the initial proposal used to choose a boundary. A batched invocation counts as one physical NFE and separately contributes one evaluated row per batch element. We do not translate NFE reductions into floating-point operation (FLOP), energy, or latency claims. Accuracy is paired on identical prompt IDs. A run with a failed manifest, incomplete coverage, a mock record, or a failed claim gate is ineligible for a positive aggregate.

F Reproducibility, compute, and assets

Configurations and statistics. The anonymous supplement contains the implementation, exact configuration files, fixed model and dataset revisions, run manifests, and artifact summaries referenced in Table 4. The initial audit uses seed 20260710; the cross-model protocols use seed 20260729. Confidence intervals are percentile intervals from 10,000 paired bootstrap resamples of prompt identifiers. The risk tests use exact binomial bounds while limiting the probability of any false selection across the tested policy family to 0.05. The supplement documents the commands for the initial

379 Strategy 1 evaluation and each RC-PAG protocol. Raw checkpoints and benchmark copies are not
380 redistributed; the loaders fetch the fixed public assets.

381 **Compute.** We estimate approximately 160 hours on an NVIDIA A100 graphics processing unit
382 (GPU), equivalent to 160 A100 GPU-hours. Trace collection and the original PAG study account
383 for about 40 hours; the six completed GPU protocols account for about 20 hours each, or 120
384 hours. Two intermediate configurations were superseded before full GPU runs, and the equivalence
385 preflight stopped before a benchmark rollout; incomplete job attempts and host-only analysis are
386 excluded. The estimate assumes one NVIDIA A100 PCIe (80 GB), with LLaDA and Dream loaded
387 one at a time in bfloat16. The recorded environment used Python 3.11.15, PyTorch 2.5.1 with
388 CUDA 12.1, Transformers 4.46.3, Datasets 4.8.5, scikit-learn 1.9.0, and math-verify 0.9.0. Atomic
389 records retain per-prompt elapsed time; the batched pilots ranged from roughly one to six seconds
390 per method/prompt.

391 **Licenses.** The pinned LLaDA checkpoint is MIT licensed and the pinned Dream checkpoint is
392 Apache-2.0 licensed. GSM8K, MATH-500, and HumanEval are used under their MIT licenses;
393 MBPP data are used under CC BY 4.0. We cite each model and benchmark at first use and preserve
394 their original terms. The supplement contains no model weights or benchmark examples beyond the
395 aggregate statistics and one short diagnostic trace shown in Figure 2.

396 **G Broader impact**

397 Reducing sequential model calls can lower latency and energy when the extra batch work does not
398 erase the gain. It can also make generative systems cheaper to deploy at scale. The experiments
399 expose the corresponding reliability risk: an efficiency controller that commits tokens too early
400 can retain attractive proxy metrics while changing answers substantially. Systems using learned
401 scheduling should therefore keep a reference fallback, audit task quality, and report batch work as
402 well as call count. This work releases no new foundation model or scraped dataset and does not
403 expand the underlying models’ capabilities; any downstream misuse or bias remains that of the base
404 models and their deployment context.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and Section 1 state the evaluated settings, efficiency results, accuracy trade-offs, failed gates, and the resulting scope of the design claim. Sections 5 and 7 support and delimit those claims.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 7 discusses model scale, benchmark scope, detector dependence, calibration scope, latency, and the limits of the failed verification audits.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper contains no theorem or formal proof. The finite-sample risk-control procedure invokes established methods and states the loss, policy family, and confidence level in Section 4.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Sections 3 and 4 specify the scheduler, features, model settings, splits, controls, accounting, gates, and statistics. The appendix and anonymous supplement provide pinned configurations, commands, manifests, and artifact provenance.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in

some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: An anonymized source supplement accompanies the submission with the implementation, exact YAML configurations, run commands, and non-mock artifact summaries. Public model and dataset assets are fetched at the pinned revisions listed in the supplement.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Architecture and training details appear in Section 3.2; decoding, dataset splits, seeds, evaluation sizes, thresholds, and policy-selection rules appear in Section 4 and the reproducibility appendix.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Section 4 defines the paired design and 10,000-sample bootstrap procedure. Section 5 reports the main paired accuracy interval and the minimum confirmatory lower bound; null or failed intervals are not presented as wins.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The compute appendix reports a total estimate of approximately 160 A100-equivalent GPU-hours: 40 hours for trace collection and the original PAG experiments, plus six completed artifact-backed protocols at approximately 20 hours each. It also reports GPU memory, execution mode, software versions, exclusions, and representative per-prompt times.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The work evaluates public models on public benchmarks, preserves double-blind anonymity, reports failed promotion gates, and does not involve private data, human subjects, or a high-risk model release.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The Broader Impact appendix discusses possible efficiency and energy benefits, lower deployment cost, reliability failures from premature commitment, scaling of underlying model harms, and reference-fallback mitigation.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper releases no new pretrained model, scraped dataset, or capability-enhancing checkpoint. The source supplement contains only decoding and evaluation code, configurations, and aggregate artifacts.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Original model and benchmark creators are cited at first use. The reproducibility appendix states the pinned model licenses (MIT and Apache-2.0) and benchmark licenses (MIT and CC BY 4.0); the supplement does not redistribute checkpoints or datasets.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The new scheduler implementation, configurations, and evaluation artifacts are documented in the anonymous supplement, including exact commands, schema-checked manifests, run hashes, and known limitations.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The research does not use crowdsourcing or human participants.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

722 **15. Institutional review board (IRB) approvals or equivalent for research with human**
723 **subjects**

724 Question: Does the paper describe potential risks incurred by study participants, whether
725 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
726 approvals (or an equivalent approval/review based on the requirements of your country or
727 institution) were obtained?

728 Answer: [N/A]

729 Justification: The research does not use crowdsourcing or human participants, so IRB re-
730 view is not applicable.

731 Guidelines:

- 732 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
733 with human subjects.
- 734 • Depending on the country in which research is conducted, IRB approval (or equiva-
735 lent) may be required for any human subjects research. If you obtained IRB approval,
736 you should clearly state this in the paper.
- 737 • We recognize that the procedures for this may vary significantly between institutions
738 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
739 guidelines for their institution.
- 740 • For initial submissions, do not include any information that would break anonymity
741 (if applicable), such as the institution conducting the review.

742 **16. Declaration of LLM usage**

743 Question: Does the paper describe the usage of LLMs if it is an important, original, or
744 non-standard component of the core methods in this research? Note that if the LLM is used
745 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
746 scientific rigor, or originality of the research, declaration is not required.

747 Answer: [N/A]

748 Justification: Generative AI tools were used only for writing, editing, and formatting. The
749 research questions, method design, implementation, experiments, evaluation, analysis, and
750 scientific interpretation were performed by the authors; no LLM generated labels or re-
751 placed experimental evaluation.

752 Guidelines:

- 753 • The answer [N/A] means that the core method development in this research does not
754 involve LLMs as any important, original, or non-standard components.
- 755 • Please refer to our LLM policy in the NeurIPS handbook for what should or should
756 not be described.